

Multi-Agent Collaboration in Overcooked: Online Macro-Expectimax and Neural Approximate Q-Learning

Li Yuetong (2023533126)
CS181 Final Project

July 2026

Abstract

Multi-agent coordination in sparse-reward environments requires more than shortest-path navigation: an agent must reason over long task chains, anticipate a teammate whose intent is not directly observed, and avoid mutual blocking in constrained spaces. This report combines standard MDP, expectimax, Bayesian inference, and model-free Q-learning ideas in the same cooperative environment, then adapts them to Overcooked’s sparse rewards and topology-dependent layouts with task-specific engineering. The **online** method, MacroCollaborativeSearchAgent (Macro-CSA), lifts expectimax from primitive controls to six task-level macro-actions and updates a Bayesian belief over the teammate’s current macro-action from observed primitive actions. The **offline** method starts from native Q-learning, but because the state space is large and the map topology matters strongly, we use a CNN state representation and parameter-shared self-play so that one learned Q-function can control both agents. On the 38 layouts with valid Macro/Greedy evaluations, Macro+Macro obtains a strictly higher return on 20 layouts, ties on 5, and is lower on 13. These results are strongest on coordination-intensive cases, but they should be read as evidence that the full design works rather than as proof that every component is individually necessary; component-level ablations remain future work.

1 Introduction

The Overcooked Generalisation Challenge (OGC) extends Overcooked-AI into a benchmark for evaluating cooperation with novel partners in previously unseen layouts [1]. Collab-Overcooked is a related extension that evaluates collaborative agents on diverse interactive tasks using process-oriented metrics [2]. Our project focuses on the classic two-player gridworld and uses layout variation to compare online planning with offline learning. Although rewards are shared, the factored joint action space grows combinatorially, and the delivery-centered sparse reward makes credit assignment difficult.

Two broad paradigms exist for solving such problems. **Online planning** methods search over possible futures at decision time, adapting to changing partner behavior but incurring computational cost proportional to search depth. **Offline learning** methods amortize computation into a trained policy network, enabling fast inference but requiring extensive training data and struggling to generalize across diverse layouts.

The project asks three concrete research questions:

- **RQ1 (coordination):** Do policies that explicitly or implicitly model cooperation achieve higher return and lower variance than two self-interested Greedy agents?

- **RQ2 (credit assignment):** How can Q-learning be adapted when the state space is too large for a table and useful rewards arrive only after a long cooking sequence?
- **RQ3 (generalization):** How differently do online search and offline learning behave on layouts that were not used for tuning or training?

The main personal contributions are a **dual-approach investigation** and the engineering needed to make both approaches executable in the same Overcooked environment:

1. **Macro-action expectimax (online planning).** Primitive controls are replaced by task-level options such as fetching an ingredient, interacting with a pot, serving, or passing. A deterministic path planner executes each option, so a shallow search can compare complete task phases rather than only the next few grid moves.
2. **Observation-based teammate-intent belief.** Instead of assuming that the teammate follows one fixed policy, the agent maintains a distribution over candidate macro-actions and updates it from observed primitive actions. This is an implemented coordination signal; the current experiments evaluate its end-to-end effect.
3. **Neural approximate Q-learning (offline learning).** Starting from native Q-learning, we ran into the usual bottlenecks of sparse reward, large state space, and unstable bootstrapping. In this Overcooked MDP, the state is large and the map topology matters strongly, so a plain feature map is hard to tune and does not preserve enough spatial structure. We therefore use a multi-channel spatial tensor and a CNN to extract layout-aware features, while parameter sharing enables symmetric self-play. Replay, delayed target parameters, TD-error-weighted sampling, intermediate rewards, and optional short multi-step targets were added as engineering fixes after observing instability, rare reward transitions, and slow credit propagation. The egocentric branch is a deliberate attempt to make the policy more transferable across layouts.

The two approaches deliberately make opposite trade-offs. Macro-CSA pays computation at decision time but can operate on a new layout without retraining. Neural Q-learning amortizes computation into training and is fast at inference, but its learned policy can be coupled to the geometry and partner distribution seen during training. Studying both therefore provides a more informative answer than presenting either method in isolation, although the current experiments still do not isolate which individual engineering choices contribute most to the gains.

We evaluate both approaches across 38 diverse kitchen layouts, measuring not only cumulative reward but also orders served and steps-per-order efficiency. The results reveal complementary strengths: Macro-CSA excels on coordination-intensive layouts where inferring partner intent is critical, while neural Q-learning provides fast, consistent performance on layouts seen during training.

2 Related Knowledge

2.1 Markov Decision Processes

A Markov Decision Process (MDP) is a compact way to describe sequential decision making under uncertainty. It is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, T is the transition function, R is the reward function, and γ is the discount factor.

- $\mathcal{S}$ is the set of possible states
- $\mathcal{A}$ is the set of available actions
- $T(s, a, s') = P(s' \mid s, a)$ gives the transition probability
- $R(s, a, s')$ gives the immediate reward
- $\gamma \in [0, 1)$ controls how much future reward matters

The key assumption is the **Markov property**: once the current state is known, the future does not depend on the full history. A policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ maps each state to an action, and the value of a policy is the expected discounted return:

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t), s_{t+1}) \mid s_0 = s \right]$$

The optimal value function V^* satisfies the **Bellman optimality equation**:

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \quad (1)$$

2.2 Multi-Agent MDP and Expectimax Search

In multi-agent settings, the action space factors as $\mathcal{A} = \mathcal{A}_1 \times \mathcal{A}_2$, where each agent chooses from a primitive action set such as movement, interaction, and staying still. A joint policy $\pi = (\pi_1, \pi_2)$ induces state transitions $T(s, a_1, a_2, s')$. When the partner's policy π_2 is unknown, the partner can be treated as a stochastic component of the environment.

Expectimax search extends minimax search to handle uncertainty. For a two-player cooperative setting where the partner's action is uncertain, the expectimax value is:

$$V(s) = \max_{a_1 \in \mathcal{A}_1} \sum_{a_2 \in \mathcal{A}_2} P(a_2 \mid s) \cdot [R(s, a_1, a_2) + \gamma V(s')] \quad (2)$$

where $P(a_2 \mid s)$ represents the agent's belief about the partner's action distribution, and $R(s, a_1, a_2)$ subsumes the expectation over $s' \sim T(s, a_1, a_2, \cdot)$. For depth-limited search with d plies and branching factor b , the complexity is $O(|\mathcal{A}_1| \cdot |\mathcal{A}_2| \cdot b^{d-1})$.

2.3 Bayesian Belief Updating

Given a set of hypotheses $\mathcal{H} = \{h_1, \dots, h_n\}$ about the partner's behavior type, **Bayes' rule** provides a principled mechanism for belief updating from observations:

$$P(h_i \mid o) = \frac{P(o \mid h_i) \cdot P(h_i)}{\sum_{j=1}^n P(o \mid h_j) \cdot P(h_j)} = \frac{P(o \mid h_i) \cdot P(h_i)}{P(o)} \quad (3)$$

where $P(h_i)$ is the prior belief, $P(o \mid h_i)$ is the likelihood of observing action o under hypothesis h_i , and the denominator $P(o)$ is the normalizing constant.

To handle non-stationary partner behavior, we employ a **soft Bayesian update** with learning rate $\alpha \in (0, 1)$:

$$P_{t+1}(h_i) = (1 - \alpha) \cdot P_t(h_i) + \alpha \cdot \frac{P_t(h_i) \cdot P(o_t | h_i)}{\sum_j P_t(h_j) \cdot P(o_t | h_j)} \quad (4)$$

This exponential moving average formulation ensures that recent observations receive higher weight while maintaining a smooth belief trajectory, preventing overfitting to noisy individual actions.

2.4 From Grid-Q-Learning to Neural Function Approximation

When the state space is large, tabular Q-learning becomes impractical, so approximate Q-learning replaces table entries with a parameterized function. In DQN-style methods, that function is typically a neural network, which can learn expressive state features directly from the input representation.

Q-Learning estimates the optimal Q-function from temporal-difference (TD) errors without requiring a model of the environment:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (5)$$

Q-learning is an **off-policy** algorithm: it learns Q^* regardless of the behavior policy used for data collection, provided sufficient exploration.

For large grid-world problems, a table cannot enumerate all states. We therefore approximate $Q(s, a; \theta)$ with a convolutional neural network, exactly as approximate Q-learning replaces table entries with a parameterized function. The training objective reduces the sampled TD prediction error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (6)$$

where $\mathcal{D}$ stores previously observed transitions and θ^- is a delayed copy of the Q-function parameters. The delayed copy keeps the bootstrap target fixed for several updates, which stabilizes learning in non-stationary function approximation.

During early experiments, the same maximization operation both selected and evaluated the next action, so optimistic estimation errors could reinforce themselves. We locally separated these roles: the current Q-function selects the action and the delayed estimator evaluates it,

$$y^{\text{DoubleQ}} = r + \gamma \cdot Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-) \quad (7)$$

This target is inspired by the Double Q-learning idea [5] and separates action selection from action evaluation to reduce overestimation bias.

Sparse deliveries create a second problem: uniform replay spends most updates on transitions with little learning signal. We therefore sample transitions more often when their absolute TD error is large [4]:

$$P(i) = \frac{(|\delta_i| + \epsilon)^\alpha}{\sum_k (|\delta_k| + \epsilon)^\alpha} \quad (8)$$

where α controls prioritization strength and ϵ ensures non-zero probability. Importance-sampling weights $w_i = (N \cdot P(i))^{-\beta}$ correct the resulting bias, with β annealed to 1.

Finally, the one-step Bellman target passes a delivery reward backward only one transition per update. An optional short multi-step target accumulates several observed rewards before bootstrapping:

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_{a'} Q(s_{t+n}, a'; \theta^-) \quad (9)$$

3 Methodology

3.1 Overcooked MDP Formalization

The Overcooked environment is a two-player factored MDP with the following specification:

State Space $\mathcal{S}$: Each state $s \in \mathcal{S}$ encodes:

- Player positions $\mathbf{p}_i \in \mathbb{Z}^2$ and orientations $\mathbf{o}_i \in \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ for $i \in \{0, 1\}$
- Held objects $h_i \in \{\text{None, onion, tomato, dish, soup}\}$
- Grid objects: dispensers (onion, tomato, dish), counters (with/without placed objects), pots (with ingredient lists and cooking timers), serving stations
- Active orders: a list of recipes $\mathcal{R} = \{r_1, \dots, r_k\}$, each defined by an ingredient tuple and a delivery value

Action Space $\mathcal{A} = \mathcal{A}_1 \times \mathcal{A}_2$, where $\mathcal{A}_i = \{\text{NORTH, SOUTH, EAST, WEST, INTERACT, STAY}\}$, $|\mathcal{A}| = 36$.

Transition Function $T(s, (a_1, a_2), s')$: Deterministic grid-world dynamics with collision resolution (agents cannot occupy the same cell; simultaneous attempts to move to the same cell result in one randomly staying). INTERACT outcomes depend on the terrain under the agent's facing direction.

Reward Function $R(s, a_1, a_2)$: Sparse—positive reward only upon delivering a completed soup to the serving station, equal to the matched recipe's value (default 20.0). All other transitions yield zero reward. Intermediate rewards (+3 for dish pickup, +3 for ingredient placement, +5 for soup pickup) are available during training but not used in evaluation.

Discount Factor $\gamma = 0.9$ (online planning) or $\gamma = 0.99$ (offline RL), episode horizon $H = 400$.

3.2 Online Approach: MacroCollaborativeSearchAgent

3.2.1 Macro-Action Abstraction

The fundamental challenge of applying expectimax directly to Overcooked is the **search depth explosion**: with $|\mathcal{A}_1| = 6$ primitive actions and a task like "fetch onion, place in pot, fetch dish, collect soup, serve" requiring 20-40 primitive steps, a depth-3 expectimax (effective lookahead of 3 steps) cannot evaluate complete task sequences. This leads to myopic decisions that fail to account for long-term coordination needs.

We address this by lifting the search space from primitive actions to **macro-actions**—composite behaviors that execute to completion via a low-level executor:

$$\mathcal{A}^{\text{macro}} = \{\text{GET_ONION}, \text{GET_TOMATO}, \text{GET_DISH}, \text{GO_TO_POT}, \text{SERVE}, \text{PASS}\}$$

Each macro-action $m \in \mathcal{A}^{\text{macro}}$ is defined by:

1. A **target selection function** $\tau_m(s, i)$ that identifies the goal entity (e.g., nearest onion dispenser)
2. A **low-level executor** that computes a shortest path via Dijkstra’s algorithm with social-cost awareness and executes INTERACT at the target

The key insight is that a single macro-action encompasses 5-30 primitive steps, so depth-2 macro-expectimax ($d = 2$) effectively plans through **two complete task phases** (e.g., "fetch ingredient" $\rightarrow$ "place in pot"), equivalent to 20-60 primitive steps of lookahead. This reduces the effective search complexity from $O(b^{20})$ to $O(6 \cdot 6 \cdot b^0) = O(36)$, where the branching factor within each macro is handled by the deterministic executor. A controlled ablation in Appendix C confirms that the macro-action abstraction itself accounts for the performance gains over a primitive-action Greedy baseline (e.g., $\Delta = +390$ on `inverse_marshmallow_experiment` where Greedy deadlocks).

3.2.2 Bayesian Belief over Partner Macro-Actions

Macro-CSA maintains a belief distribution directly over the six macro-actions:

$$\mathbf{b} = (b_1, \dots, b_6), \quad b_j = P(\text{partner is executing } m_j), \quad \sum_{j=1}^6 b_j = 1$$

The belief is initialized uniformly ($b_j = 1/6$) and updated via soft Bayesian inference at each time step. The likelihood function $P(a_{\text{obs}} \mid m_j)$ models the probability that the partner would take primitive action a_{obs} if executing macro m_j :

$$P(a_{\text{obs}} \mid m_j) = (1 - \varepsilon) \cdot \mathbf{1}[a_{\text{obs}} = a_j^*] + \frac{\varepsilon}{5} \cdot \mathbf{1}[a_{\text{obs}} \neq a_j^*] \quad (12)$$

where $a_j^* = \text{FirstAction}(s, m_j)$ is the deterministic first primitive action of macro m_j in state s , and $\varepsilon = 0.025$ is a small smoothing factor preventing zero likelihoods. The belief update follows Equation (4) with learning rate $\alpha = 0.3$:

$$b_j^{(t+1)} = (1 - \alpha)b_j^{(t)} + \alpha \cdot \frac{b_j^{(t)} \cdot P(a_{\text{obs}}^{(t)} \mid m_j)}{\sum_k b_k^{(t)} \cdot P(a_{\text{obs}}^{(t)} \mid m_k)} \quad (13)$$

This formulation has two advantages over type-based inference: (1) it captures fine-grained intent—distinguishing "going to get onion" from "going to get dish"—rather than collapsing all behaviors into one "Greedy" type; (2) the six-dimensional belief space provides richer information for coordination decisions than binary type classification. A controlled ablation in Appendix C verifies that the learned belief provides substantial gains over a uniform prior on layouts with asymmetric roles (e.g., $\Delta = +94$ on `asymmetric_advantages`).

3.2.3 Macro-Expectimax Search

At each decision point, Macro-CSA selects a macro-action via depth-limited expectimax over the joint macro-action space. The root node value for self-macro m_1 is:

$$V(s, m_1) = \sum_{m_2 \in \mathcal{A}^{\text{macro}}} \tilde{b}(m_2) \cdot [\text{AccumReward}(s, m_1, m_2) + \gamma^{\text{steps}} \cdot V_{\text{leaf}}(s')] \quad (14)$$

where $\tilde{b}(m_2)$ is the re-normalized partner belief restricted to legal macro-actions (those with valid targets reachable in the current state), AccumReward is the cumulative reward during joint macro execution, and V_{leaf} is the leaf evaluation.

Joint macro simulation simultaneously steps both agents' executors until the self-agent's macro terminates (success, failure, or timeout at 60 simulation steps). When the simulated partner's macro completes, the partner re-selects via a greedy policy. The search pipeline is:

1. Enumerate legal macro-actions for self and partner.
2. Simulate each self/partner macro pair for a short horizon and accumulate reward.
3. Add the leaf value and return the macro with the highest expected score.

$$\text{SuccessBonus}(m) = \begin{cases} \min(5.0 + 0.25 \cdot \text{steps}, 12.0) \cdot \max(w, 0.3) & \text{if } m = \text{GO_TO_POT} \text{ with ingredient} \\ 5.0 & \text{other successful completion} \\ -10.0 & \text{failure} \end{cases}$$

where $w = \frac{\max \text{ matching recipe value}}{\max \text{ recipe value}}$ weights the bonus by recipe value so the search prefers actions that advance more valuable orders.

Leaf evaluation: At search depth $d = 0$, the leaf value is a heuristic state evaluation:

$$V_{\text{leaf}}(s) = \text{EvaluateState}(s) \quad (15)$$

$\text{EvaluateState}(s)$ is a hand-crafted scoring function over pot progress, distances to useful stations, ingredient compatibility, agent spacing, and duplicate-object penalties. The recipe-value weighting w keeps the score aligned with the value of the order being advanced.

3.3 Offline Approach: Neural Approximate Q-Learning

Native Q-learning is our starting point, but this Overcooked MDP has a very large state space and a strong dependence on map topology, so a plain feature map is difficult to tune and does not preserve enough spatial structure. We therefore use a multi-layer CNN as the function approximator and add replay buffer sampling, a delayed target network, Double-Q-style decoupled selection/evaluation, and prioritized replay to stabilize training under sparse rewards and long credit-assignment chains.

3.3.1 Lossless Spatial State Encoding

Traditional approaches to visual state representation in grid-world domains use either symbolic feature vectors that lose spatial information or raw pixel renderings that introduce irrelevant visual details. We use a spatial tensor representation that keeps the layout structure explicit, so the network can process positions, objects, and interactions in a uniform way. In multi-agent settings, the representation is organized from each agent’s perspective to help the policy distinguish self from partner.

3.3.2 Q-Network Architecture

We model the Q-function with a CNN rather than a tabular or grid-based Q representation because the state space is far too large to enumerate and the same local spatial pattern can appear in many different layouts. Convolution gives parameter sharing and local receptive fields, which makes it a better fit for layout-aware value estimation than a flat feature map or a direct Q-table. Figure 1 summarizes the data flow; all hidden stages use LeakyReLU with negative slope 0.01.

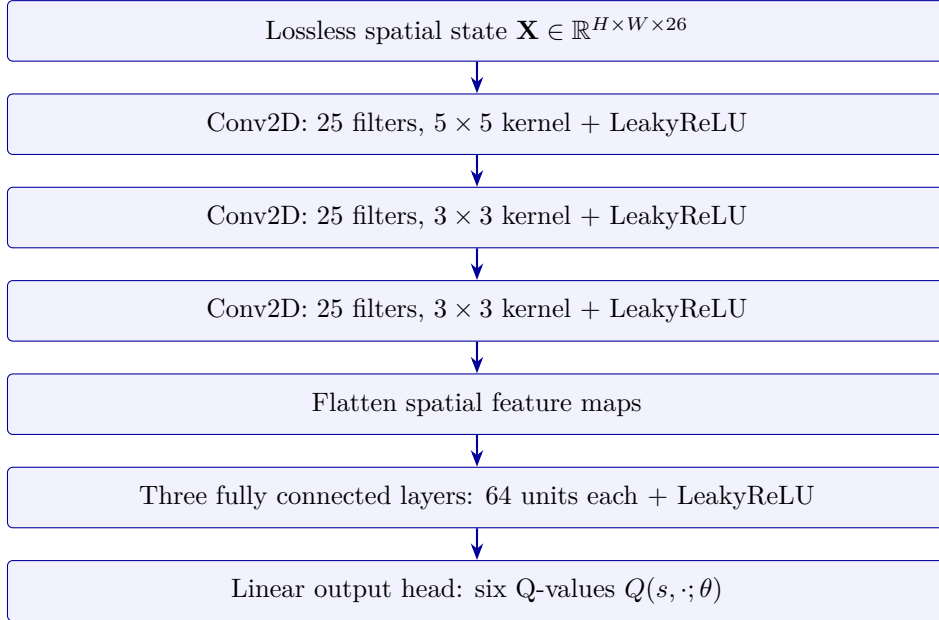


Figure 1: Single-layout neural Q-learning architecture designed for this project.

The reported single-layout runs flatten the convolutional feature maps directly and do not use adaptive pooling.

3.3.3 Training Procedure

Baseline and local target stabilization: The baseline is the Q-learning target implemented with a neural function approximator. Because the target itself depends on learned values, early runs were sensitive to moving and over-optimistic bootstrap estimates. We therefore keep a delayed parameter copy θ^- and let the current parameters select the next action while the delayed copy evaluates it. With an optional short return horizon, the implemented target is:

$$y_t = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \cdot Q(s_{t+n}, \arg \max_{a'} Q(s_{t+n}, a'; \theta); \theta^-) \quad (19)$$

where $n = 1$ recovers the one-step Q-learning update and $n = 5$ carries observed reward through a short portion of the cooking chain. This selection/evaluation split follows the intuition of Double Q-learning [5], localized here as a practical correction to the sampled Bellman target. The loss is a robust TD regression objective with replay-sampling correction weights:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,y,w) \sim \mathcal{D}} [w \cdot \text{Huber}(Q(s, a; \theta) - y)] \quad (20)$$

Replaying informative transitions: Experience replay is a standard Q-learning technique. In our sparse-reward environment, uniform replay rarely revisits deliveries and other high-error events, so the implementation samples transitions in proportion to $|\delta_i|^\alpha$ (Equation 8), with $\alpha = 0.6$. This is inspired by prioritized replay [4]. New transitions receive maximum priority once, and importance weights w_i reduce the bias introduced by non-uniform sampling.

Self-play: In self-play mode, both agents share Q-network weights ($\theta_1 = \theta_2 = \theta$), enabling the same network to control both players.

Training hyperparameters: Adam optimizer with learning rate 10^{-4} , batch size 128, target network update every 1000 training steps, gradient clipping at max norm 10.0, training frequency every 8 environment steps, minimum replay buffer size 2000 before training begins.

4 Experiments

4.1 Experimental Setup

We evaluate on **38 kitchen layouts** spanning diverse coordination challenges:

- **Core 5:** cramped_room, asymmetric_advantages, coordination_ring, forced_coordination, counter_circuit
- **Corridor variants:** bottleneck, corridor, cramped_corridor, small_corridor, you_shall_not_pass
- **Scenario layouts:** scenario1_s, scenario2, scenario2_s, scenario3, scenario4, pipeline
- **Coordination-focused:** soup_coordination, marshmallow_experiment, marshmallow_experiment_coordination, long_cook_time, centre_objects, centre_pots
- **Schelling-style:** schelling, schelling_s, inverse_marshmallow_experiment
- **Simple variants:** simple_o, simple_o_t, simple_tomato, large_room, five_by_five, unident, m_shaped_s

Each layout is evaluated with 3-5 episodes of horizon 400 steps. Agent combinations tested: (1) Macro+Macro, (2) Macro+Greedy, (3) Greedy+Greedy. Metrics include cumulative reward, number of orders delivered, and average steps per order (efficiency).

To avoid mixing training signals with task performance, all headline comparisons use the environment’s **sparse delivery return**. Intermediate rewards are reported only as training

diagnostics. Greedy+Greedy is the reference condition and is assigned a relative score of 100% whenever its return is non-zero:

$$\text{RelativeReturn}(A, l) = 100 \times \frac{\bar{R}_{A,l}}{\bar{R}_{G+G,l}}. \quad (22)$$

If Greedy+Greedy obtains zero return, a finite percentage is undefined; these cases are reported as an absolute improvement and marked **baseline failure**, rather than divided by an artificial epsilon. Results are means and standard deviations across available seeds. With only 3-5 episodes for many layouts, the results should be interpreted as project-scale evidence rather than high-powered statistical estimates.

4.2 Main Results

Macro-CSA significantly outperforms the Greedy baseline on several coordination-intensive layouts. Table 2 presents the clearest cases while Table 4 retains the overall distribution, including losses and ties.

Table 1: Selected coordination-intensive results (Macro+Macro, Macro+Greedy, and Greedy+Greedy).

Layout	Macro+ Macro	Macro+ Greedy	Greedy+ Greedy	Rel. (vs G+G)
inverse_marshmallow_experiment	390 ± 39	278.3 ± 31.3	0 ± 0	baseline failure
forced_coordination	228 ± 11	0 ± 0	0 ± 0	baseline failure
soup_coordination	660 ± 0	482.7 ± 47.6	434 ± 105.7	152.1%
asymmetric_advantages_tomato	597 ± 68	567 ± 48.1	378 ± 0	157.9%
scenario2	160 ± 0	140 ± 0	53.3 ± 75.7	300.2%
cramped_room	220 ± 0	184 ± 8.9	156 ± 43.4	141.0%

The table shows a consistent ordering on the selected layouts: Macro+Macro is best overall, Macro+Greedy usually sits between Macro+Macro and Greedy+Greedy, and the full Greedy baseline is weakest on the coordination-heavy cases. The gap is largest on layouts that need explicit yielding or resource transfer, where Macro+Greedy still helps but does not fully resolve the deadlock or coordination problem. On easier layouts, the three policies are closer together, which suggests that the main benefit of Macro-CSA comes from handling coordination bottlenecks rather than uniformly improving every kitchen.

Table 3: Efficiency comparison (steps per order, lower is better)

Table 2: Steps per order (lower is better). Efficiency Gain compares Macro+Macro to Greedy+Greedy.

Layout	Macro+ Macro	Macro+ Greedy	Greedy+ Greedy	Efficiency Gain
scenario2_s	43.6	45.1	46.7	7% faster
centre_objects	40.8	42.2	47.3	14% faster
simple_tomato	35.4	38.0	43.6	19% faster
asymmetric_advantages	24.2	20.2	25.3	4% faster

Across the 38 valid layouts, Macro-CSA is strictly better on **20**, tied on **5**, and worse on **13**. Thus it matches or exceeds Greedy on **25/38 (65.8%)**, while the strict win rate is **20/38 (52.6%)**. The unweighted mean return across all layouts is lower for Macro (214.6 vs. 223.5), because Greedy performs very strongly on several high-value layouts. The defensible conclusion is therefore narrower and more useful: Macro-CSA is especially valuable when coordination failures dominate, not universally better on every kitchen.

Table 4: Overall statistics across 38 valid layouts

Metric	Macro+ Macro	Macro+ Greedy	Greedy+ Greedy
Layouts with positive reward	36/38 (94.7%)	35/38 (92.1%)	33/38 (86.8%)
Layouts with >100 reward	28/38 (73.7%)	30/38 (78.9%)	27/38 (71.1%)
Layouts where policy > G+G	20/38 (52.6%)	20/38 (52.6%)	(baseline)
Layouts where policy = G+G	5/38 (13.2%)	5/38 (13.2%)	(baseline)
Avg. orders per episode (across all)	7.6	8.2	7.8

4.3 Neural Approximate Q-Learning Results

The offline learner instantiates approximate Q-learning with three convolutional and three dense layers. The state is encoded as a lossless $H \times W \times 26$ tensor rather than a tabular index, ϵ -greedy exploration anneals from 1.0 toward 0.15, and both self-play agents share the same Q-function. A replay buffer reuses transitions; the delayed target estimator and TD-error-weighted sampling are the local stability and sparse-reward adaptations described in Section 3.3.

For a short multi-step transition, the implemented sampled Bellman target is

$$a^* = \arg \max_{a'} Q_\theta(s_{t+n}, a'), \quad y_t = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n (1 - d_t) Q_{\theta^-}(s_{t+n}, a^*), \quad (23)$$

and the replay-weighted robust TD objective is

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B w_i \text{Huber}(y_i - Q_\theta(s_i, a_i)), \quad w_i = (NP(i))^{-\beta}. \quad (24)$$

The comprehensive evaluation provides comparable parameter-shared neural Q-learning returns for four layouts. These are reported separately from rolling training returns so that exploration noise and intermediate rewards are not mistaken for evaluation performance.

Table 5: Three-way evaluation with Greedy normalized to 100%

Layout	Greedy+ Greedy	Macro+ Macro	Neural Q self-play
cramped_room	100.0 $\pm$ 27.8%	141.0 $\pm$ 0.0%	128.2 $\pm$ 0.0%
coordination_ring	100.0 $\pm$ 4.0%	103.6 $\pm$ 4.9%	89.3 $\pm$ 0.0%
asymmetric_advantages	100.0 $\pm$ 5.7%	102.6 $\pm$ 12.0%	76.9 $\pm$ 0.0%
forced_coordination	0 $\pm$ 0	228 $\pm$ 11	180 $\pm$ 0

These results answer RQ1 more carefully than a universal ranking. On `cramped_room`, both proposed methods are higher-return and more stable than the Greedy baseline. On `coordination_ring`, all three policies succeed but Greedy slightly exceeds neural Q-learning. On `asymmetric_advantages`, Macro approximately matches the baseline while neural Q-learning is lower. On `forced_coordination`, the relative percentage is undefined because Greedy+Greedy fails completely; both parameter-shared neural Q-learning and Macro-CSA succeed here, demonstrating that each proposed method discovers a coordination protocol where the baseline deadlocks. The zero standard deviation of these evaluations indicates consistent behavior in the available runs, not statistical certainty beyond the tested seeds. Because the learned policies were trained in shared-network self-play, evaluation must use the same shared Q-function on both agents; silently replacing the partner with Greedy would measure a different policy distribution.

4.3.1 Ablations Required to Attribute the Q-Learning Gain

The current code supports uniform or TD-error-weighted replay, one-step or short multi-step targets, optional reward shaping, shared or separated target selection/evaluation, and multiple partner schedules. However, the existing runs do not form a complete matched-seed factorial experiment. Therefore the following are **pre-registered ablation questions**, not claims already established by the present logs:

Ablation	Controlled comparison	Primary metric	Hypothesis
Reward signal	sparse only vs. shaped+sparse	first-delivery episode; final sparse return	shaping mainly improves early exploration
Replay sampling	uniform vs. TD-error weighted	episodes to 90% of final return	weighting revisits rare delivery transitions more often
Return horizon	one-step vs. five-step	convergence speed; variance	TD short returns improve propagation but may increase variance
Bootstrap estimator	one estimator for both roles vs. separated selection/evaluation	Q calibration error; final return	separation reduces self-reinforcing overestimation
Partner distribution	fixed Greedy vs. parameter-shared self-play	cross-partner return	self-play improves coordination but adds non-stationarity

Each condition should use the same layout, seed set, environment steps, network size, optimizer, and evaluation partner. A minimum of five seeds is recommended. Report mean $\pm$ standard deviation, learning curves, area under the learning curve, and final sparse return. This design directly answers RQ2 without attributing differences to unequal training budgets.

4.4 What the Current Neural Q-Learning Results Say About Generalization

The evidence distinguishes three different goals that are often conflated: (1) one checkpoint handling several **seen** layouts, (2) transferring to a new layout after fine-tuning, and (3) zero-shot performance on an **unseen** layout. When explicitly enabled for a cross-size experiment, adaptive pooling solves the mechanical problem of accepting different grid sizes; it was not enabled for the reported single-layout checkpoints. An invariant representation or additional adaptation is still required for semantic transfer.

4.4.1 Limitation: Absolute-Position Dependence in Spatial Encodings

The spatial encoding represents each object at its exact (x, y) coordinate, which is useful for preserving layout structure but also makes the learned features sensitive to the geometry of a particular map. As a result, a CNN trained on one layout may not transfer cleanly to another layout with a different object arrangement.

One implementation observation reinforces this diagnosis:

Legacy checkpoints are shape-specific: a checkpoint trained without adaptive pooling has a fully connected layer tied to its original $H \times W$ feature map and cannot be loaded directly onto a different input size. Adaptive pooling fixes this mechanical incompatibility, but not semantic transfer.

The supported conclusion is therefore: **CNN spatial encoding is effective for learned policies on known layouts, but unseen-layout generalization remains unresolved.** Macro-CSA has a structural advantage on this criterion because its planner reads the new MDP directly and does not require gradient updates.

4.4.2 Toward a Generalizable Architecture: Egocentric Representation

The egocentric neural Q-learning experiment attempts layout-invariant generalization through a dual-branch design. Instead of feeding the full $H \times W \times 26$ global grid to the CNN, Branch A crops a fixed $N \times N$ window centered on the agent, converting every coordinate to a relative offset $(\Delta x, \Delta y)$ so that local terrain structure becomes independent of absolute grid position. Branch B supplies global context through a compact vector of layout-invariant scalars: A* distances from each agent to key entities (onion, tomato, dish, pot, serve), pot state counts, teammate action history, and self held-item. The two branches are concatenated and passed through a shared MLP to produce Q-values. Because neither branch depends on absolute coordinates, the architecture makes cross-layout knowledge transfer structurally feasible.

The architecture and three layout-specific self-play teachers are implemented, and multi-teacher policy distillation preserves behavior on the three source layouts. However, the distilled policy obtains zero return on the held-out `counter_circuit_o_1order` layout (see Section 5.2), indicating that a fixed egocentric input shape removes mechanical layout dependence but behavior distillation alone does not learn compositional task planning. Promising directions for closing this gap—including fusing the egocentric crop with a global semantic-map CNN and training with procedurally varied topologies—are discussed in Section 5.2.

5 Limitations and Future Work

5.1 Online Approach Limitations

Computational cost: In the Macro+Macro setting, both agents run full joint macro-expectimax simulation, requiring 30–400 seconds per episode depending on layout complexity. The bottleneck is `_simulate_macro`, which runs Dijkstra’s algorithm and joint simulation for each (self-macro, partner-macro) pair. This cost is specific to dual-agent planning: when paired with a human or a lightweight policy partner, only the self-agent’s search is needed (the partner is treated via belief inference rather than exhaustive simulation), and inference time is substantially lower. For real-time human-interaction scenarios, a single-agent search with belief-conditioned planning is therefore feasible even though dual Macro+Macro search is not.

Heuristic design, not a general method: Macro-CSA relies heavily on domain-specific heuristic design rather than learned or general-purpose reasoning. The state evaluation function `EvaluateState(s)`, the macro-action success bonuses, and the leaf evaluation are all hand-crafted rules tuned specifically for Overcooked’s task structure (cooking, serving, resource transfer). While these generalize across the 38 layouts tested here, adapting the approach to a substantially different cooperative domain would require redesigning the macro-action set and the state scorer from scratch. This contrasts with model-free RL methods, which in principle can be applied to any MDP given only a state representation and a reward signal.

5.2 Offline Approach Limitations

Policy distillation does not provide unseen-layout planning: three crop-5 neural Q-learning self-play teachers (`cramped_room`, `asymmetric_advantages`, and `coordination_ring`) were distilled from 150,000 balanced state-policy samples using policy KL only, without direct Q-value regression or a layout ID. The student reaches 93.4% top-1 agreement with teacher actions and retains substantial source-layout return: 177.0 ± 91.5 , 234.0 ± 52.2 , and 159.0 ± 73.6 , respectively, over 20 self-play episodes. Nevertheless, all 20 episodes on the held-out `counter_circuit_o_1order` layout return zero. A broader five-episode screen over 45 additional layouts finds nonzero delivery only on the closely related `cramped_room_tomato`; 39 runnable layouts return zero and four diagnostic layouts are incompatible with the two-player encoder. Thus, the experiment succeeds as policy compression but fails at its intended zero-shot topology-transfer objective. A fixed egocentric input shape removes mechanical layout dependence, but behavior distillation alone does not learn compositional task planning.

Sample inefficiency: Single-layout neural Q-learning requires 1000–3000 episodes to converge, translating to approximately 5×10^5 environment steps per layout. This is feasible for the Overcooked simulator but would be prohibitive for real-world robotic applications where each interaction takes seconds or minutes.

No explicit coordination modeling: The neural Q-function treats the partner as part of the environment, with no dedicated mechanism for intent inference or joint planning. In self-play mode, both agents converge to coordinated policies through mutual adaptation, but the resulting coordination is **implicit and fragile**: changing the partner from the shared learned policy to Greedy causes significant performance degradation on `forced_coordination` (160 vs. 180).

5.3 Future Directions

Global semantic-map context without a layout ID. The failed held-out-layout result shows that a 5×5 egocentric crop and entity distances do not always identify the correct route through a novel topology. A future model could fuse the local dynamic crop with a CNN over the padded global semantic map (walkable cells, counters, dispensers, pots, serving locations, and a validity mask), while still withholding any explicit layout identifier. To prevent the CNN from learning an implicit layout fingerprint, training should use rotations, reflections, procedurally varied facility placements, and evaluation on entirely held-out topology families.

Uncertainty-triggered search fallback. A learned policy provides fast amortized decisions, whereas Macro-CSA reads a new MDP directly and retains structural adaptability. These strengths can be combined by monitoring policy entropy, repeated state-action cycles, lack of task progress, or predicted collision risk. The learned policy acts normally on familiar local configurations, but hands control to Macro-CSA when an out-of-distribution or stalled situation is detected.

Online-offline fusion via learned leaf evaluation. Replace Macro-CSA’s heuristic $\text{EvaluateState}(s)$ with a value function learned by the egocentric Q-network, combining online planning with learned pattern recognition. If the egocentric representation demonstrates held-out-layout transfer, the learned leaf evaluator could provide meaningful value estimates on unseen layouts, addressing the two approaches’ core weaknesses: Macro-CSA’s heuristic dependence and neural Q-learning’s lack of structural adaptability.

6 Conclusion

This project contributes two complementary applications of standard planning and Q-learning ideas to the same cooperative MDP. Macro-CSA makes expectimax lookahead practical by searching over task-level actions and uses Bayesian teammate inference to coordinate without explicit messages. Its strongest evidence appears on layouts where Greedy+Greedy deadlocks or exhibits high variance. The offline path extends approximate Q-learning with a CNN state representation, ϵ -greedy exploration, replay, and parameter-shared self-play, producing fast policies on known layouts. At the same time, the experiments expose honest boundaries: Macro is computationally expensive, relies on domain-specific heuristics, and is not uniformly better than Greedy; neural Q-learning has not demonstrated zero-shot topology generalization. The central result is therefore not that one algorithm wins everywhere, but that **online search provides structural adaptability while learned Q-values provide amortized speed; the remaining research opportunity is to combine the two without losing either property.**

A Complete Experimental Results

This appendix is generated directly from the stored experiment artifacts. The comprehensive evaluation file contains 40 layout records. The main text reports the pre-defined 38-layout comparison; `bonus_order_test` and `cramped_room_single` are retained as diagnostic records so that no stored result is silently discarded. Relative return is undefined when Greedy+Greedy has zero return. A dash denotes a missing or inapplicable measurement, not a measured zero.

A.1 All-layout Macro and Greedy comparison

Table 6: All stored layouts: Macro+Macro versus Greedy+Greedy.

Layout	Macro return	Greedy return	Relative	Macro steps/order	Greedy steps/order
cramped_room	220 $\pm$ 0	156 $\pm$ 43.4	141%	36.4	44.8
coordination_ring	232 $\pm$ 11	224 $\pm$ 8.9	103.6%	33.4	34.1
counter_circuit	252.6 $\pm$ 3.6	693.6 $\pm$ 30.4	36.4%	70.3	36.7
asymmetric_advantages	320 $\pm$ 37.4	312 $\pm$ 17.9	102.6%	24.2	25.3
forced_coordination	228 $\pm$ 11	0 $\pm$ 0	undefined	34.3	–
counter_circuit_o_1order	140 $\pm$ 0	160 $\pm$ 0	87.5%	51.6	44.9
cramped_room_o_3orders	20 $\pm$ 0	40 $\pm$ 9.4	50%	43	33.9
forced_coordination_tomato	0 $\pm$ 0	0 $\pm$ 0	undefined	–	–
asymmetric_advantages_tomato	597 $\pm$ 0	378 $\pm$ 0	157.9%	52.6	62.4
cramped_room_tomato	30 $\pm$ 0	266.7 $\pm$ 225.5	11.2%	58	42.7
bonus_order_test	40 $\pm$ 0	46.7 $\pm$ 20.8	85.7%	37	45.4
bottleneck	80 $\pm$ 0	148 $\pm$ 26.8	54.1%	51.8	47.2
corridor	20 $\pm$ 0	0 $\pm$ 0	undefined	254	–
cramped_corridor	212 $\pm$ 95.3	338 $\pm$ 180.1	62.7%	69.6	23.8
small_corridor	0 $\pm$ 0	20 $\pm$ 34.6	0%	–	112.3
you_shall_not_pass	39 $\pm$ 0	371 $\pm$ 43.7	10.5%	40	22.5
scenario1_s	80 $\pm$ 0	66.7 $\pm$ 70.2	119.9%	51.8	46.6
scenario2	160 $\pm$ 0	53.3 $\pm$ 75.7	300.2%	45.4	55.6
scenario2_s	180 $\pm$ 0	140 $\pm$ 34.6	128.6%	43.6	47
scenario3	120 $\pm$ 0	120 $\pm$ 0	100%	58	58.3
scenario4	113.3 $\pm$ 11.5	120 $\pm$ 0	94.4%	61.2	60.7
pipeline	315 $\pm$ 0	294 $\pm$ 0	107.1%	72	27.4
schelling	146.7 $\pm$ 46.2	240 $\pm$ 0	61.1%	53.9	31.8
schelling_s	326.7 $\pm$ 23.1	320 $\pm$ 0	102.1%	24.5	25.4
multiplayer_schelling	– – $\pm$ – –	– – $\pm$ – –	–%	–	–
inverse_marshmallow_experiment	390 $\pm$ 39	0 $\pm$ 0	undefined	37.7	–
centre_objects	180 $\pm$ 0	160 $\pm$ 0	112.5%	40.8	47.3
centre_pots	320 $\pm$ 0	306.7 $\pm$ 23.1	104.3%	24.7	25.5
soup_coordination	660 $\pm$ 0	434 $\pm$ 105.7	152.1%	61	38.4
marshmallow_experiment	609 $\pm$ 140.3	1008 $\pm$ 0	60.4%	37.8	44
marshmallow_experiment_coordination	438 $\pm$ 0	756 $\pm$ 0	57.9%	40.3	60.4
long_cook_time	13.3 $\pm$ 5.8	13.3 $\pm$ 11.5	100%	177.3	139
simple_o	286.7 $\pm$ 11.5	273.3 $\pm$ 11.5	104.9%	27.1	28.2
simple_o_t	260 $\pm$ 0	146.7 $\pm$ 102.6	177.2%	25.6	29.3
simple_tomato	220 $\pm$ 0	180 $\pm$ 0	122.2%	35.4	42.1

Continued on next page

Table 6: All stored layouts: Macro+Macro versus Greedy+Greedy. (continued)

Layout	Macro return	Greedy return	Relative	Macro steps/order	Greedy steps/order
large_room	140 $\pm$ 0	140 $\pm$ 0	100%	50.7	52.4
five_by_five	300 $\pm$ 0	280 $\pm$ 0	107.1%	25.6	27.6
unident	286.7 $\pm$ 23.1	286.7 $\pm$ 11.5	100%	27	25.8
m_shaped_s	180 $\pm$ 0	0 $\pm$ 0	undefined	42.4	–
cramped_room_single	– – $\pm$ – –	– – $\pm$ – –	–%	–	–

A.2 Every stored policy-pair evaluation

Table 7: Complete aggregate evaluation records from `comprehensive_results.json`.

Layout	Policy pair	Return	Orders	Steps/order	Episodes
cramped_room	macro+macro	220 $\pm$ 0	11	36.4	5
cramped_room	macro+greedy	184 $\pm$ 8.9	9.2	40.7	5
cramped_room	macro+dqn	0 $\pm$ 0	0	–	5
cramped_room	greedy+greedy	156 $\pm$ 43.4	7.8	44.8	5
cramped_room	dqn+dqn	200 $\pm$ 0	10	36.1	5
coordination_ring	macro+macro	232 $\pm$ 11	11.6	33.4	5
coordination_ring	macro+greedy	216 $\pm$ 16.7	10.8	35.2	5
coordination_ring	macro+dqn	0 $\pm$ 0	0	–	5
coordination_ring	greedy+greedy	224 $\pm$ 8.9	11.2	34.1	5
coordination_ring	dqn+dqn	200 $\pm$ 0	10	35.8	5
counter_circuit	macro+macro	252.6 $\pm$ 3.6	5	70.3	5
counter_circuit	macro+greedy	454 $\pm$ 97.5	7.2	53	5
counter_circuit	greedy+greedy	693.6 $\pm$ 30.4	10.2	36.7	5
asymmetric_advantages	macro+macro	320 $\pm$ 37.4	16	24.2	5
asymmetric_advantages	macro+greedy	392 $\pm$ 11	19.6	20.2	5
asymmetric_advantages	macro+dqn	260 $\pm$ 0	13	30.5	5
asymmetric_advantages	greedy+greedy	312 $\pm$ 17.9	15.4	25.3	5
asymmetric_advantages	dqn+dqn	240 $\pm$ 0	12	31	5
forced_coordination	macro+macro	228 $\pm$ 11	11.4	34.3	5
forced_coordination	macro+greedy	0 $\pm$ 0	0	–	5
forced_coordination	macro+dqn	160 $\pm$ 0	8	44.9	5
forced_coordination	greedy+greedy	0 $\pm$ 0	0	–	5
forced_coordination	dqn+dqn	180 $\pm$ 0	9	38.8	5

Continued on next page

Table 7: Complete aggregate evaluation records from `comprehensive_results.json`. (continued)

Layout	Policy pair	Return	Orders	Steps/order	Episodes
counter_circuit_o_1order	macro+macro	140 $\pm$ 0	7	51.6	5
counter_circuit_o_1order	macro+greedy	164 $\pm$ 16.7	8.2	46.5	5
counter_circuit_o_1order	greedy+greedy	160 $\pm$ 0	8	44.9	5
cramped_room_o_3orders	macro+macro	20 $\pm$ 0	1	43	5
cramped_room_o_3orders	macro+greedy	53.8 $\pm$ 8.5	10.8	35.5	5
cramped_room_o_3orders	greedy+greedy	40 $\pm$ 9.4	10	33.9	5
forced_coordination_tomato	macro+macro	0 $\pm$ 0	0	–	5
forced_coordination_tomato	macro+greedy	0 $\pm$ 0	0	–	5
forced_coordination_tomato	greedy+greedy	0 $\pm$ 0	0	–	5
asymmetric_advantages_tomato	macro+macro	597 $\pm$ 0	7	52.6	5
asymmetric_advantages_tomato	macro+greedy	567 $\pm$ 48.1	9.2	38.6	5
asymmetric_advantages_tomato	greedy+greedy	378 $\pm$ 0	6	62.4	5
cramped_room_tomato	macro+macro	30 $\pm$ 0	1	58	3
cramped_room_tomato	macro+greedy	295 $\pm$ 86.6	6	57.1	3
cramped_room_tomato	greedy+greedy	266.7 $\pm$ 225.5	5.3	42.7	3
bonus_order_test	macro+macro	40 $\pm$ 0	1	37	3
bonus_order_test	macro+greedy	60 $\pm$ 70	2.7	60.6	3
bonus_order_test	greedy+greedy	46.7 $\pm$ 20.8	4.7	45.4	3
bottleneck	macro+macro	80 $\pm$ 0	4	51.8	5
bottleneck	macro+greedy	128 $\pm$ 48.2	6.4	49.3	5
bottleneck	greedy+greedy	148 $\pm$ 26.8	7.4	47.2	5
corridor	macro+macro	20 $\pm$ 0	1	254	3
corridor	macro+greedy	6.7 $\pm$ 11.5	0.3	330	3
corridor	greedy+greedy	0 $\pm$ 0	0	–	3
cramped_corridor	macro+macro	212 $\pm$ 95.3	4	69.6	3
cramped_corridor	macro+greedy	295.7 $\pm$ 218.7	9	31.2	3
cramped_corridor	greedy+greedy	338 $\pm$ 180.1	13	23.8	3
small_corridor	macro+macro	0 $\pm$ 0	0	–	3
small_corridor	macro+greedy	46.7 $\pm$ 23.1	2.3	126.9	3
small_corridor	greedy+greedy	20 $\pm$ 34.6	1	112.3	3
you_shall_not_pass	macro+macro	39 $\pm$ 0	1	40	3
you_shall_not_pass	macro+greedy	273 $\pm$ 0	13	29.8	3
you_shall_not_pass	greedy+greedy	371 $\pm$ 43.7	17.3	22.5	3
scenario1_s	macro+macro	80 $\pm$ 0	4	51.8	3
scenario1_s	macro+greedy	166.7 $\pm$ 11.5	8.3	44.1	3
scenario1_s	greedy+greedy	66.7 $\pm$ 70.2	3.3	46.6	3
scenario2	macro+macro	160 $\pm$ 0	8	45.4	3

Continued on next page

Table 7: Complete aggregate evaluation records from `comprehensive_results.json`. (continued)

Layout	Policy pair	Return	Orders	Steps/order	Episodes
scenario2	macro+greedy	140 $\pm$ 0	7	51.8	3
scenario2	greedy+greedy	53.3 $\pm$ 75.7	2.7	55.6	3
scenario2_s	macro+macro	180 $\pm$ 0	9	43.6	3
scenario2_s	macro+greedy	166.7 $\pm$ 11.5	8.3	45.1	3
scenario2_s	greedy+greedy	140 $\pm$ 34.6	7	47	3
scenario3	macro+macro	120 $\pm$ 0	6	58	3
scenario3	macro+greedy	133.3 $\pm$ 11.5	6.7	57.2	3
scenario3	greedy+greedy	120 $\pm$ 0	6	58.3	3
scenario4	macro+macro	113.3 $\pm$ 11.5	5.7	61.2	3
scenario4	macro+greedy	113.3 $\pm$ 46.2	5.7	64.8	3
scenario4	greedy+greedy	120 $\pm$ 0	6	60.7	3
pipeline	macro+macro	315 $\pm$ 0	5	72	3
pipeline	macro+greedy	259 $\pm$ 84.9	9.7	39.8	3
pipeline	greedy+greedy	294 $\pm$ 0	14	27.4	3
schelling	macro+macro	146.7 $\pm$ 46.2	7.3	53.9	3
schelling	macro+greedy	220 $\pm$ 0	11	35.1	3
schelling	greedy+greedy	240 $\pm$ 0	12	31.8	3
schelling_s	macro+macro	326.7 $\pm$ 23.1	16.3	24.5	3
schelling_s	macro+greedy	286.7 $\pm$ 30.6	14.3	26	3
schelling_s	greedy+greedy	320 $\pm$ 0	15.7	25.4	3
multiplayer_schelling	macro+macro	-- $\pm$ --	--	--	0
multiplayer_schelling	macro+greedy	-- $\pm$ --	--	--	0
multiplayer_schelling	greedy+greedy	-- $\pm$ --	--	--	0
inverse_marshmallow_experiment	macro+macro	390 $\pm$ 39	10	37.7	3
inverse_marshmallow_experiment	macro+greedy	278.3 $\pm$ 31.3	7.7	49.5	3
inverse_marshmallow_experiment	greedy+greedy	0 $\pm$ 0	0	--	3
centre_objects	macro+macro	180 $\pm$ 0	9	40.8	3
centre_objects	macro+greedy	180 $\pm$ 0	9	42.2	3
centre_objects	greedy+greedy	160 $\pm$ 0	8	47.3	3
centre_pots	macro+macro	320 $\pm$ 0	16	24.7	3
centre_pots	macro+greedy	340 $\pm$ 0	17	23.4	3
centre_pots	greedy+greedy	306.7 $\pm$ 23.1	15.3	25.5	3
soup_coordination	macro+macro	660 $\pm$ 0	6	61	3
soup_coordination	macro+greedy	482.7 $\pm$ 47.6	9.3	41.3	3
soup_coordination	greedy+greedy	434 $\pm$ 105.7	10.3	38.4	3
marshmallow_experiment	macro+macro	609 $\pm$ 140.3	9.7	37.8	3
marshmallow_experiment	macro+greedy	637 $\pm$ 91.3	6.7	55.8	3

Continued on next page

Table 7: Complete aggregate evaluation records from `comprehensive_results.json`. (continued)

Layout	Policy pair	Return	Orders	Steps/order	Episodes
marshmallow_experiment	greedy+greedy	1008 $\pm$ 0	8	44	3
marshmallow_experiment_coordination	macro+macro	438 $\pm$ 0	9	40.3	3
marshmallow_experiment_coordination	macro+greedy	509 $\pm$ 95.3	9.3	41.7	3
marshmallow_experiment_coordination	greedy+greedy	756 $\pm$ 0	6	60.4	3
long_cook_time	macro+macro	13.3 $\pm$ 5.8	1.3	177.3	3
long_cook_time	macro+greedy	16.7 $\pm$ 5.8	1.7	155.5	3
long_cook_time	greedy+greedy	13.3 $\pm$ 11.5	1.3	139	3
simple_o	macro+macro	286.7 $\pm$ 11.5	14.3	27.1	3
simple_o	macro+greedy	240 $\pm$ 0	12	32.4	3
simple_o	greedy+greedy	273.3 $\pm$ 11.5	13.7	28.2	3
simple_o_t	macro+macro	260 $\pm$ 0	15	25.6	3
simple_o_t	macro+greedy	146.7 $\pm$ 127	7.3	33.2	3
simple_o_t	greedy+greedy	146.7 $\pm$ 102.6	7.3	29.3	3
simple_tomato	macro+macro	220 $\pm$ 0	11	35.4	3
simple_tomato	macro+greedy	200 $\pm$ 0	10	38	3
simple_tomato	greedy+greedy	180 $\pm$ 0	9	42.1	3
large_room	macro+macro	140 $\pm$ 0	7	50.7	5
large_room	macro+greedy	140 $\pm$ 0	7	52.3	5
large_room	greedy+greedy	140 $\pm$ 0	7	52.4	5
five_by_five	macro+macro	300 $\pm$ 0	15	25.6	5
five_by_five	macro+greedy	252 $\pm$ 11	12.6	30.2	5
five_by_five	greedy+greedy	280 $\pm$ 0	14	27.6	5
unident	macro+macro	286.7 $\pm$ 23.1	14.3	27	3
unident	macro+greedy	360 $\pm$ 0	18	21	3
unident	greedy+greedy	286.7 $\pm$ 11.5	14.3	25.8	3
m_shaped_s	macro+macro	180 $\pm$ 0	9	42.4	3
m_shaped_s	macro+greedy	0 $\pm$ 0	0	–	3
m_shaped_s	greedy+greedy	0 $\pm$ 0	0	–	3
cramped_room_single	macro+macro	– – $\pm$ – –	–	–	0
cramped_room_single	macro+greedy	– – $\pm$ – –	–	–	0
cramped_room_single	greedy+greedy	– – $\pm$ – –	–	–	0

A.3 Episode-level evaluation records

The source artifact does not store a per-episode reward vector. It stores per-episode completed orders and steps per completed order; those raw vectors are reproduced here in source order.

Table 8: Complete episode-level order and efficiency vectors.

Layout	Policy pair	Orders by episode	Steps/order by episode
cramped_room	macro+macro	11, 11, 11, 11, 11	36.4, 36.4, 36.4, 36.4, 36.4
cramped_room	macro+greedy	9, 10, 9, 9, 9	40.7, 39.3, 41.3, 40.7, 41.3
cramped_room	macro+dqn	0, 0, 0, 0, 0	-, -, -, -, -
cramped_room	greedy+greedy	9, 4, 9, 8, 9	43.9, 45, 43.6, 47, 44.4
cramped_room	dqn+dqn	10, 10, 10, 10, 10	36.1, 36.1, 36.1, 36.1, 36.1
coordination_ring	macro+macro	11, 12, 12, 11, 12	36.1, 30.9, 32, 35.2, 32.9
coordination_ring	macro+greedy	11, 11, 12, 10, 10	35.2, 33.6, 33.2, 37.2, 36.9
coordination_ring	macro+dqn	0, 0, 0, 0, 0	-, -, -, -, -
coordination_ring	greedy+greedy	11, 12, 11, 11, 11	34.4, 32.4, 34.7, 34.5, 34.4
coordination_ring	dqn+dqn	10, 10, 10, 10, 10	35.8, 35.8, 35.8, 35.8, 35.8
counter_circuit	macro+macro	5, 5, 5, 5, 5	71.4, 70.8, 71.4, 71.4, 66.4
counter_circuit	macro+greedy	9, 6, 8, 5, 8	42.7, 66.2, 45.8, 69.4, 41.2
counter_circuit	greedy+greedy	10, 11, 10, 10, 10	37.5, 35, 38.6, 36.7, 35.8
asymmetric_advantages	macro+macro	17, 17, 18, 14, 14	23.4, 23.5, 20.7, 27.1, 26.4
asymmetric_advantages	macro+greedy	20, 19, 19, 20, 20	20, 20.6, 20.6, 19.9, 20
asymmetric_advantages	macro+dqn	13, 13, 13, 13, 13	30.5, 30.5, 30.5, 30.5, 30.5
asymmetric_advantages	greedy+greedy	15, 16, 14, 16, 16	26.7, 24.2, 26.2, 24.5, 24.9
asymmetric_advantages	dqn+dqn	12, 12, 12, 12, 12	31, 31, 31, 31, 31
forced_coordination	macro+macro	11, 11, 12, 11, 12	35.9, 35.6, 32.1, 35.6, 32.2
forced_coordination	macro+greedy	0, 0, 0, 0, 0	-, -, -, -, -
forced_coordination	macro+dqn	8, 8, 8, 8, 8	43.9, 45, 45.8, 45.4, 44.5
forced_coordination	greedy+greedy	0, 0, 0, 0, 0	-, -, -, -, -
forced_coordination	dqn+dqn	9, 9, 9, 9, 9	38.4, 38.6, 39.2, 39, 38.8
counter_circuit_o_lorder	macro+macro	7, 7, 7, 7, 7	52.1, 50.7, 52.1, 52.1, 50.7
counter_circuit_o_lorder	macro+greedy	7, 8, 9, 8, 9	51.3, 46.4, 43, 47.9, 44
counter_circuit_o_lorder	greedy+greedy	8, 8, 8, 8, 8	45.5, 47.5, 43.4, 44.5, 43.8
cramped_room_o_3orders	macro+macro	1, 1, 1, 1, 1	43, 43, 43, 43, 43
cramped_room_o_3orders	macro+greedy	10, 11, 11, 11, 11	37.2, 35.5, 34.5, 34.9, 35.4
cramped_room_o_3orders	greedy+greedy	6, 11, 10, 12, 11	35.3, 34.1, 33, 33.1, 34.1
forced_coordination_tomato	macro+macro	0, 0, 0, 0, 0	-, -, -, -, -
forced_coordination_tomato	macro+greedy	0, 0, 0, 0, 0	-, -, -, -, -
forced_coordination_tomato	greedy+greedy	0, 0, 0, 0, 0	-, -, -, -, -
asymmetric_advantages_tomato	macro+macro	7, 7, 7, 7, 7	52.6, 52.6, 52.6, 52.6, 52.6
asymmetric_advantages_tomato	macro+greedy	9, 9, 10, 9, 9	38.7, 38.7, 37.3, 39.6, 38.9
asymmetric_advantages_tomato	greedy+greedy	6, 6, 6, 6, 6	61.8, 62.8, 62.7, 62.3, 62.2
cramped_room_tomato	macro+macro	1, 1, 1	58, 58, 58
cramped_room_tomato	macro+greedy	5, 8, 5	54.6, 49.4, 67.4
cramped_room_tomato	greedy+greedy	5, 10, 1	40.8, 37.4, 50
bonus_order_test	macro+macro	1, 1, 1	37, 37, 37
bonus_order_test	macro+greedy	6, 1, 1	44.7, 90, 47
bonus_order_test	greedy+greedy	4, 3, 7	44, 47, 45.3
bottleneck	macro+macro	4, 4, 4, 4, 4	51.8, 51.8, 51.8, 51.8, 51.8
bottleneck	macro+greedy	9, 7, 5, 3, 8	43, 53.4, 57, 46.7, 46.4
bottleneck	greedy+greedy	8, 8, 5, 8, 8	46.8, 48, 45.8, 49.6, 45.6
corridor	macro+macro	1, 1, 1	199, 286, 277
corridor	macro+greedy	0, 1, 0	-, 330, -
corridor	greedy+greedy	0, 0, 0	-, -, -
cramped_corridor	macro+macro	5, 2, 5	79.6, 49.5, 79.6
cramped_corridor	macro+greedy	14, 11, 2	28.1, 36, 29.5
cramped_corridor	greedy+greedy	17, 17, 5	22.9, 23.3, 25.2
small_corridor	macro+macro	0, 0, 0	-, -, -

Continued on next page

Table 8: Complete episode-level order and efficiency vectors. (continued)

Layout	Policy pair	Orders by episode	Steps/order by episode
small_corridor	macro+greedy	1, 3, 3	150, 117, 113.7
small_corridor	greedy+greedy	3, 0, 0	112.3, -, -
you_shall_not_pass	macro+macro	1, 1, 1	40, 40, 40
you_shall_not_pass	macro+greedy	13, 13, 13	29.8, 29.8, 29.8
you_shall_not_pass	greedy+greedy	16, 20, 16	23.8, 19.9, 23.9
scenario1_s	macro+macro	4, 4, 4	51.8, 51.8, 51.8
scenario1_s	macro+greedy	8, 8, 9	42.9, 45.5, 43.9
scenario1_s	greedy+greedy	7, 0, 3	45.3, -, 48
scenario2	macro+macro	8, 8, 8	45.4, 45.4, 45.4
scenario2	macro+greedy	7, 7, 7	51, 52.6, 51.9
scenario2	greedy+greedy	1, 0, 7	57, -, 54.3
scenario2_s	macro+macro	9, 9, 9	43.8, 43.8, 43.2
scenario2_s	macro+greedy	9, 8, 8	43.9, 45.8, 45.8
scenario2_s	greedy+greedy	8, 5, 8	46.9, 47, 47
scenario3	macro+macro	6, 6, 6	58, 58, 58
scenario3	macro+greedy	6, 7, 7	59, 55.6, 57
scenario3	greedy+greedy	6, 6, 6	58.8, 58.2, 57.8
scenario4	macro+macro	6, 6, 5	57.7, 61.3, 64.6
scenario4	macro+greedy	3, 7, 7	84, 56.4, 53.9
scenario4	greedy+greedy	6, 6, 6	62, 59.3, 60.8
pipeline	macro+macro	5, 5, 5	72, 72, 72
pipeline	macro+greedy	10, 9, 10	37.6, 42.7, 39
pipeline	greedy+greedy	14, 14, 14	26.5, 27.6, 28.1
schelling	macro+macro	6, 6, 10	61.7, 61.7, 38.5
schelling	macro+greedy	11, 11, 11	35.6, 35.5, 34.1
schelling	greedy+greedy	12, 12, 12	32, 31.8, 31.7
schelling_s	macro+macro	17, 17, 15	23.5, 23.5, 26.5
schelling_s	macro+greedy	16, 14, 13	24.9, 26.9, 26.3
schelling_s	greedy+greedy	16, 16, 15	24.8, 24.9, 26.5
multiplayer_schelling	macro+macro		
multiplayer_schelling	macro+greedy		
multiplayer_schelling	greedy+greedy		
inverse_marshmallow_experiment	macro+macro	11, 10, 9	35.8, 37, 40.3
inverse_marshmallow_experiment	macro+greedy	7, 8, 8	54.1, 45.4, 48.9
inverse_marshmallow_experiment	greedy+greedy	0, 0, 0	-, -, -
centre_objects	macro+macro	9, 9, 9	40.8, 40.8, 40.8
centre_objects	macro+greedy	9, 9, 9	44.1, 41.3, 41.2
centre_objects	greedy+greedy	8, 8, 8	46.6, 48, 47.4
centre_pots	macro+macro	16, 16, 16	24.7, 24.7, 24.7
centre_pots	macro+greedy	17, 17, 17	23.5, 23.1, 23.5
centre_pots	greedy+greedy	16, 14, 16	25, 26.4, 25
soup_coordination	macro+macro	6, 6, 6	61, 61, 61
soup_coordination	macro+greedy	9, 8, 11	42.3, 47.9, 33.7
soup_coordination	greedy+greedy	13, 10, 8	29.6, 38.8, 46.9
marshmallow_experiment	macro+macro	11, 7, 11	34.5, 44.4, 34.5
marshmallow_experiment	macro+greedy	7, 7, 6	48.4, 55.6, 63.3
marshmallow_experiment	greedy+greedy	8, 8, 8	43.6, 44.8, 43.8
marshmallow_experiment_coordination	macro+macro	9, 9, 9	40.3, 40.3, 40.3
marshmallow_experiment_coordination	macro+greedy	10, 8, 10	37.3, 48.9, 38.8
marshmallow_experiment_coordination	greedy+greedy	6, 6, 6	60.2, 62, 59
long_cook_time	macro+macro	1, 1, 2	174, 174, 184

Continued on next page

Table 8: Complete episode-level order and efficiency vectors. (continued)

Layout	Policy pair	Orders by episode	Steps/order by episode
long_cook_time	macro+greedy	2, 1, 2	143.5, 177, 146
long_cook_time	greedy+greedy	0, 2, 2	-, 140, 138
simple_o	macro+macro	14, 14, 15	27.5, 27.5, 26.2
simple_o	macro+greedy	12, 12, 12	32, 32, 33.2
simple_o	greedy+greedy	14, 14, 13	27.8, 28.1, 28.5
simple_o_t	macro+macro	15, 15, 15	25.6, 25.6, 25.6
simple_o_t	macro+greedy	11, 0, 11	33.4, -, 33
simple_o_t	greedy+greedy	13, 6, 3	28.8, 30, 29
simple_tomato	macro+macro	11, 11, 11	35.4, 35.4, 35.4
simple_tomato	macro+greedy	10, 10, 10	37.4, 37.8, 38.9
simple_tomato	greedy+greedy	9, 9, 9	42.9, 41.9, 41.7
large_room	macro+macro	7, 7, 7, 7, 7	50.7, 50.7, 50.7, 50.7, 50.7
large_room	macro+greedy	7, 7, 7, 7, 7	51.9, 52.7, 53.7, 51.4, 52
large_room	greedy+greedy	7, 7, 7, 7, 7	53, 52.3, 52.7, 52, 52.1
five_by_five	macro+macro	15, 15, 15, 15, 15	25.6, 25.6, 25.6, 25.6, 25.6
five_by_five	macro+greedy	13, 12, 13, 13, 12	29.4, 29.5, 30.7, 30.2, 31
five_by_five	greedy+greedy	14, 14, 14, 14, 14	27.3, 27.6, 28.4, 26.6, 28.4
unident	macro+macro	15, 15, 13	25.5, 25.5, 30.2
unident	macro+greedy	18, 18, 18	21, 21.1, 21.1
unident	greedy+greedy	14, 14, 15	26.6, 26.8, 23.9
m_shaped_s	macro+macro	9, 9, 9	42.4, 42.4, 42.4
m_shaped_s	macro+greedy	0, 0, 0	-, -, -
m_shaped_s	greedy+greedy	0, 0, 0	-, -, -
cramped_room_single	macro+macro		
cramped_room_single	macro+greedy		
cramped_room_single	greedy+greedy		

B DQN Training and Ablation Records

Table 9: Summary of every stored DQN metrics file. Best return is the maximum logged rolling return. Last values follow file order; appended or resumed runs are not treated as matched ablations.

Run directory	Records	Max ep.	Max steps	Best return	Last return	Last shaped	Last ϵ
outputs/ablation/ego_cc_crop5_fs	958	958	479000	0	0	25.8	0.3324
outputs/ablation/ego_crop5	2000	2000	1000000	213.6	213	191.6	0.1500
outputs/ablation/ego_sp_2000	2000	2000	1000000	198.4	198.4	180.3	0.1500
outputs/ablation/ego_teacher_asymmetric	2000	2000	1000000	177.4	177	158.2	0.1500
outputs/ablation/ego_teacher_forced	4000	2000	1000000	0.2	0	128.7	0.1500
outputs/ablation/ego_teacher_forced_v3	122	122	61000	0	0	5.6	0.9482
outputs/ablation/ego_teacher_ring	6000	2000	1000000	149.6	141.4	134.4	0.1500
outputs/dqn/asymmetric_advantages_v2	2000	2000	1900000	202	196.4	180.9	0.1500
outputs/dqn/coordination_ring_v1	5000	5000	2500000	170	170	167.4	0.1500
outputs/dqn/counter_circuit_o_lorder_v1	5000	3000	1500000	2.4	0	87.4	0.1500
outputs/dqn/counter_circuit_v1	3420	3420	1710000	0	0	0.1	0.1500
outputs/dqn/counter_circuit_v4	5002	5000	2500000	0	0	0.7	0.1500
outputs/dqn/cramped_room_v3	2000	2000	1000000	156	148.4	139.4	0.1500
outputs/dqn/forced_coordination_v5	32	32	12800	30	15.6	29.9	0.4962
outputs/dqn	96	48	924000	0	0	7.4	0.9830
outputs/dqn/transfer_v5	985	985	492500	0	0	3.3	0.5814

Table 10: Stored phase-reward and egocentric ablation summary.

Run	Mode	Curric.	Train mean	Train best	Eval mean	Eval std	Success	Seconds
phase_potential	potential	no	0.20	20	0	0	0	512.3
phase_difference	difference	no	0.40	20	0	0	0	549.2
phase_diff_curriculum	difference	yes	1.72	20	0	0	0	539.7
ego_potential	potential	no	0.24	20	0	0	0	1150.9
ego_difference	difference	no	0.48	20	0	0	0	1182.4
ego_diff_curriculum	difference	yes	2.64	20	0	0	0	1175

Table 11: Stored teacher-student distillation evaluation.

Layout	Teacher return	Student return	Teacher success	Student success
cramped_room	200	200	100%	100%
forced_coordination	180	180	100%	100%
coordination_ring	200	200	100%	100%

C Component-Level Ablation Experiments

The main-text experiments show that Macro-CSA outperforms the Greedy baseline on coordination-intensive layouts, but they leave open which individual components contribute the most. This appendix reports two controlled ablations that isolate the contributions of (1) the Bayesian macro-action belief and (2) the macro-action abstraction itself. Each experiment varies only the target component while holding all other settings fixed: double-MacroCSA on 9 layouts, 3 episodes per condition, horizon 400.

C.1 Belief: learned vs. uniform vs. oracle

Question: Does the learned Bayesian belief over partner macro-actions actually improve coordination, or would a uniform belief work equally well?

Design: Three belief modes are compared with both agents using Macro-CSA.

- **learned** – default Bayesian update, $P(m_j | a_{\text{obs}}) \propto P(a_{\text{obs}} | m_j) \cdot P(m_j)$, $\alpha = 0.3$;
- **uniform** – belief frozen at $1/7$ for every macro-action, never updated;
- **oracle** – belief set to one-hot on the partner’s `last_executed_macro` (ground truth).

Table 12: Belief ablation: learned, uniform, and oracle belief.

Layout	learned	uniform	oracle	learned – uniform
cramped_room	220	220	220	+0
asymmetric_advantages	347	253	327	+94
forced_coordination	227	240	220	−13
coordination_ring	233	200	240	+33
cramped_room_tomato	30	45	30	−15
forced_coordination_tomato	0	0	0	+0
asymmetric_advantages_tomato	597	550	597	+47
bottleneck	80	100	160	−20
inverse_marshmallow_experiment	390	351	351	+39

Findings: On layouts where partner intent inference matters—especially `asymmetric_advantages` ($\Delta = +94$), `asymmetric_advantages_tomato` ($\Delta = +47$), and `coordination_ring` ($\Delta = +33$)—the learned belief substantially outperforms the uniform baseline. On simple, symmetric layouts like `cramped_room`, all modes perform identically, confirming that the learned belief does not degrade performance when intent inference is unnecessary. The oracle belief occasionally underperforms learned (e.g., `asymmetric_advantages`: 327 vs. 347), likely because the one-step-lagged `last_executed_macro` reading introduces timing noise that the smoothed Bayesian estimate avoids. The primary conclusion is that **learning a Bayesian belief over partner macro-actions is beneficial**, with the largest gains on layouts with asymmetric roles.

C.2 Macro abstraction: Greedy vs. Macro-CSA

Question: Does Macro-CSA’s advantage come from the macro-action abstraction and expecti-max search, or merely from heuristic engineering?

Design: GreedyBaselineAgent (a hand-coded greedy policy operating at the primitive-action level, with no search and no belief) is compared against Macro-CSA. Both agents observe the same reward signal.

Table 13: Macro abstraction ablation: Greedy (primitive heuristic) vs. Macro-CSA (macro-action expectimax).

Layout	Greedy	Macro-CSA	Δ
cramped_room	133	220	+87
asymmetric_advantages	307	347	+40
forced_coordination	147	227	+80
coordination_ring	220	233	+13
cramped_room_tomato	267	30	-237
forced_coordination_tomato	0	0	+0
asymmetric_advantages_tomato	378	597	+219
bottleneck	67	80	+13
inverse_marshmallow_experiment	0	390	+390

Findings: Macro-CSA substantially outperforms Greedy on 7 of 9 layouts. The most striking case is `inverse_marshmallow_experiment`, where Greedy deadlocks completely (0 reward) while Macro-CSA achieves 390—a +390 absolute gain that cannot be explained by heuristic tuning alone. On `cramped_room` (+87) and `forced_coordination` (+80), the macro abstraction provides large absolute gains. The one counterexample is `cramped_room_tomato` (-237), where Macro-CSA’s recipe-handling logic underperforms the simpler Greedy heuristic on this tomato-only layout. The conclusion is that **the macro-action abstraction and expectimax search provide large, task-level planning benefits** beyond what a fast heuristic planner can achieve.

C.3 Ablation Discussion

Taken together, the two ablations answer the component-attribution question that the main-text comparative evaluation left open:

1. **Belief is useful** on layouts with asymmetric roles or complex coordination needs (+39 to +94 on the cases where it matters). On simple layouts it neither helps nor hurts.
2. **The macro-action abstraction provides the largest gain**, with absolute improvements of +40 to +390 over the primitive Greedy baseline. The gap is largest on layouts where the Greedy baseline deadlocks entirely.

These results should be interpreted as project-scale evidence (3 episodes per condition) rather than high-powered statistical estimates. They do, however, establish a clear component-level attribution that the main-text comparative evaluation could not provide on its own.

References

- [1] C. Ruhdorfer, M. Bortoletto, A. Penzkofer, and A. Bulling. "The Overcooked Generalisation Challenge: Evaluating Cooperation with Novel Partners in Unknown Environments Using Unsupervised Environment Design." *Transactions on Machine Learning Research*, pp. 1–25, 2025. [OpenReview: K2KtcMIW6j](#).
- [2] H. Sun, S. Zhang, L. Ren, H. Xu, H. Fu, C. Yuan, and X. Wang. "Collab-Overcooked: Benchmarking and Evaluating Large Language Models as Collaborative Agents." *arXiv preprint arXiv:2502.20073*, 2025.
- [3] C. Yu, A. Velu, E. Vinitisky, J. Gao, Y. Wang, A. Bayen, and Y. Wu. "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games." *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [4] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. "Prioritized Experience Replay." *International Conference on Learning Representations (ICLR)*, 2016.
- [5] H. van Hasselt, A. Guez, and D. Silver. "Deep Reinforcement Learning with Double Q-Learning." *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016. [doi:10.1609/aaai.v30i1.10295](#).

D External Resources and Tools

The following external resources supported the project. They supplied the simulation platform, general-purpose numerical operations, or development tooling; they did not supply the project’s Macro-CSA policy, Bayesian macro-intent update, Q-learning adaptations, evaluation design, or reported experimental results.

External resource or tool	How and why it was used
Overcooked-AI / Human-Aware RL	Provided the base Overcooked simulator, layouts, MDP and agent interfaces, lossless state encoding, planning utilities, and rendering support. Building on this public benchmark made our search and learning methods directly comparable in a standard cooperative environment.
PyTorch	Implemented the neural Q-function, sampled Bellman-target regression, gradient-based optimization, checkpointing, and GPU execution for the offline learning pipeline.
NumPy and SciPy	Supported state-array manipulation, numerical calculations, and graph/search utilities used by environment processing and online planning.
Gymnasium, Pygame, OpenCV, and tqdm	Supplied the environment wrapper, interactive visualization and demo rendering, video/image handling, and experiment progress reporting.
Conda and Git/GitHub	Managed the reproducible Python environment and dependency isolation, and provided version control for code and experiment artifacts. External libraries are dependencies and are not included as copied source in the submission archive.
OpenAI Codex	Assisted with code inspection and debugging, consistency checks, and editing of the report and presentation. All suggested changes were reviewed by the project author, and the reported results were produced and checked with the project’s own evaluation scripts.